

Amazon ML Challenge 2026 — Team B Playbook

LightGBM Modeling, Macro F_{0.5} Optimization & Official Submissions

Amazon ML Challenge 2026 — Team B Playbook

Focus Area: LightGBM Training, Macro F_{0.5} Optimization & Submissions

Operating Window: Friday 26th September (Day 1 Baseline) & Saturday 27th September (Full Optimization)

Target Milestone: Produce valid leaderboard submissions and final verified

`<team_name>_submission.zip`.

1. Executive Summary & Team Mission

Team B takes the candidate feature parquets created by Team A and delivers the competitive results. Your core mission:

- Day 1 Fast Baseline:** Submit a 100% valid TSV to the Unstop portal by 3:00 PM on Sept 26 to test formatting and establish leaderboard ranking.
- Train Gradient Boosting (LightGBM):** Fast, robust, handles class imbalance, requires no GPU.
- Optimize Decision Threshold for Macro $MF_{0.5}$:** Standard 0.5 threshold fails because $MF_{0.5}$ weights precision 2x over recall. Sweep to find the peak (typically 0.65 to 0.80).
- Master Singleton Logic:** Predict empty lists for entities without strong matches (scores a full 1.0).
- Pass Official Validator:** Zero syntax rejections using `utils/validate_submission.py`.
- Package Final Zip:** Meet all directory requirements and complete `Documentation_template.md`.

2. Team B Member Roles & Schedule

Member Role	Primary Focus	Key Responsibilities
Member 3 (Model Specialist)	LightGBM & Metrics	Model hyperparameter tuning, scale_pos_weight, custom Macro F _{0.5} evaluator
Member 4 (MLOps & Submission)	Formatting & Packaging	Test inference, threshold sweep, validator script, methodology document & zip packaging

Timeline for Sept 26 & 27

- Sept 26 (02:00 PM - 04:00 PM):** Receive 50k sample from Team A. Submit **Day 1 Baseline** to Unstop.
- Sept 26 (04:00 PM - 08:00 PM):** Build and test the custom Macro $MF_{0.5}$ metric script and threshold sweep logic.
- Sept 26 (10:00 PM):** Confirm Team A has uploaded the full parquets to `s3://<bucket>/processed/full/`.

- **Sept 27 (09:00 AM - 12:00 PM):** Train LightGBM on the full training dataset.
- **Sept 27 (12:00 PM - 03:00 PM):** Optimal threshold tuning on validation set \$
 ightarrow\$ Generate test predictions \$
 ightarrow\$ Submit run 1.
- **Sept 27 (03:00 PM - 07:00 PM):** Feature importance analysis, ensemble or hyperparameter tune \$
 ightarrow\$ Submit run 2 & 3.
- **Sept 27 (07:00 PM - 10:00 PM):** Final validation, fill `Documentation_template.md`, build `<team_name>_submission.zip`, final submit!

3. Step-by-Step Implementation Guide

Step 1: Train LightGBM with Class Imbalance Weighting

Because true matches represent only ~5-10% of generated candidate pairs, use `scale_pos_weight` to prevent the model from predicting all negatives:

```
import pandas as pd
import lightgbm as lgb

# 1. Load Parquet files from S3
train_df = pd.read_parquet("s3://<your-bucket>/processed/full/train_candidates.parquet")
val_df = pd.read_parquet("s3://<your-bucket>/processed/full/val_candidates.parquet")

features = [
    'name_ratio', 'name_partial', 'name_token_sort', 'name_token_set',
    'addr_ratio', 'addr_partial', 'addr_token_set',
    'name_len_diff', 'addr_len_diff'
]

X_train, y_train = train_df[features], train_df['label']
X_val, y_val = val_df[features], val_df['label']

# 2. Configure LightGBM Classifier
model = lgb.LGBMClassifier(
    n_estimators=600,
    learning_rate=0.05,
    num_leaves=31,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=3.0,
    random_state=42
)

model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    callbacks=[lgb.early_stopping(stopping_rounds=40)]
)
```

Step 2: Implement Macro $F_{0.5}$ & Sweep Thresholds

The competition evaluates per-Source-1 macro-average $F_{0.5}$. False merges are penalized heavily:

```
import numpy as np

def compute_f05(precision, recall):
```

```

        if (0.25 * precision + recall) == 0:
            return 0.0
        return (1.25 * precision * recall) / (0.25 * precision + recall)

val_probs = model.predict_proba(X_val)[: , 1]

best_threshold = 0.50
best_f05 = 0.0

# Sweep threshold from 0.40 to 0.92
for t in np.arange(0.40, 0.92, 0.04):
    preds = (val_probs >= t).astype(int)
    tp = np.sum((preds == 1) & (y_val == 1))
    fp = np.sum((preds == 1) & (y_val == 0))
    fn = np.sum((preds == 0) & (y_val == 1))

    prec = tp / (tp + fp) if (tp + fp) > 0 else 0
    rec = tp / (tp + fn) if (tp + fn) > 0 else 0
    score = compute_f05(prec, rec)

    print(f"Threshold: {t:.2f} | Precision: {prec:.3f} | Recall: {rec:.3f} | F_0.5: {score:.4f}")
    if score > best_f05:
        best_f05 = score
        best_threshold = t

print(f"\nWINNING THRESHOLD: {best_threshold:.2f} (Validation F_0.5 = {best_f05:.4f})")

```

Step 3: Format Test Outputs (All 1,732,544 S1 Entities)

You must generate `output/matching_results.tsv` and `output/candidate_pairs.tsv`.
Every single entity in `test_source1.tsv` must have exactly one row:

```

# Predict probabilities for test candidate pairs
test_df = pd.read_parquet("s3://<your-bucket>/processed/full/test_candidates.parquet")
test_df['prob'] = model.predict_proba(test_df[features])[:, 1]
test_df['is_match'] = test_df['prob'] >= best_threshold

# Group matches and candidates
matches_map = (
    test_df[test_df['is_match']]
    .groupby('source1_entity_id')['candidate_entity_id']
    .apply(lambda ids: ", ".join(list(dict.fromkeys(ids))))
    .to_dict()
)

candidates_map = (
    test_df
    .groupby('source1_entity_id')['candidate_entity_id']
    .apply(lambda ids: ", ".join(list(dict.fromkeys(ids))))
    .to_dict()
)

# Read all required test S1 IDs
test_s1 = pd.read_csv("dataset/test/test_source1.tsv", sep="\t")
all_s1_ids = test_s1['entity_id'].tolist()

matching_rows = []
candidate_rows = []

for s1_id in all_s1_ids:
    matching_rows.append({
        'source1_entity_id': s1_id,

```

```

        'matched_entity_ids': matches_map.get(s1_id, "")
    })
    candidate_rows.append({
        'source1_entity_id': s1_id,
        'candidate_entity_ids': candidates_map.get(s1_id, "")
    })

pd.DataFrame(matching_rows).to_csv("output/matching_results.tsv", sep="\t", index=False)
pd.DataFrame(candidate_rows).to_csv("output/candidate_pairs.tsv", sep="\t", index=False)
print("TSV files created successfully!")

```

Step 4: Validate Before Submitting (Zero Rejections)

Run this check in your terminal:

```

python3 utils/validate_submission.py \
    --matching output/matching_results.tsv \
    --candidate output/candidate_pairs.tsv \
    --test-dir dataset/test

```

Only upload to the portal when it prints: **PASS – no blocking issues found. Safe to submit.**

Step 5: Final Submission Package Assembly

Before the deadline, build the final zip:

```

zip -r my_team_submission.zip \
    output/matching_results.tsv \
    output/candidate_pairs.tsv \
    code/ \
    docs/Documentation_template.md

```

4. Team B Golden Rules & Anti-Mistakes

- **DO NOT** use default 0.5 threshold. $F_{0.5}$ rewards precision; your optimal threshold will usually be 0.65–0.78.
- **DO NOT** miss any S1 entity. All 1,732,544 IDs from `test_source1.tsv` must appear.
- **DO NOT** submit without running `utils/validate_submission.py` locally.
- **DO NOT** upload a zipped file directly to the leaderboard portal. The portal takes only `matching_results.tsv`. The zip is for final code review.